

Make Graphs Great Again: Rethinking LLM Agent Architecture via Structure–Semantic Separation

Jingyi Jia
Huazhong University of Science and
Technology
Wuhan, China
jingyijia@hust.edu.cn

Jiale Wang
Huazhong University of Science and
Technology
Wuhan, China
chia.le@foxmail.com

Haoyi Zhang
Huazhong University of Science and
Technology
Wuhan, China
haoyizhang@hust.edu.cn

Qinbin Li*
Huazhong University of Science and
Technology
Wuhan, China
qinbin@hust.edu.cn

Xiaofei Liao
Huazhong University of Science and
Technology
Wuhan, China
xfliao@hust.edu.cn

Hai Jin
Huazhong University of Science and
Technology
Wuhan, China
hjin@hust.edu.cn

Abstract

LLM-centric agent architectures route each observation back to the model for the next decision, leaving control logic implicit in a growing prompt and forcing repeated reconstruction of task state and routine execution progress even when runtime evidence determines the next transition.

We present **structure–semantic separation** as an execution boundary for LLM agents, making recurring tool-use structure explicit and executable while reserving the model for semantic decisions that structure cannot resolve. MotifAgent realizes this boundary through motif graphs: the Structure Engine advances graph transitions only when runtime evidence verifies them, while the Semantic Engine handles unresolved semantic gaps through scoped LLM calls, turning structure from prompt guidance into executable runtime state. Exposing motif progress to the serving backend additionally enables structure-aware scheduling and fine-grained KV-cache retention. Across APIBank, TauBench, and AppWorld, MotifAgent reduces LLM calls and token consumption while maintaining comparable task performance. Repeated APIBank serving replays show complementary reductions in prefill-stage latency and improvements in admission-time KV reuse under the evaluated queuing and cache-pressure conditions.

*Qinbin Li is the corresponding author. All authors are with the National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, Hubei, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, Hong Kong

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2818-1/2026/11

<https://doi.org/10.1145/3828523.3857104>

CCS Concepts: • Computing methodologies → Artificial intelligence; • Software and its engineering → Software architectures.

Keywords: LLM agents, agent runtime, LLM serving, KV cache

ACM Reference Format:

Jingyi Jia, Jiale Wang, Haoyi Zhang, Qinbin Li, Xiaofei Liao, and Hai Jin. 2026. Make Graphs Great Again: Rethinking LLM Agent Architecture via Structure–Semantic Separation. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026)*, November 15–18, 2026, Hong Kong, Hong Kong. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3828523.3857104>

1 Introduction

Enabled by the powerful abilities of Large Language Models (LLMs), LLM agents [31, 45, 49, 53, 56] are increasingly used in various domains such as coding, web automation, and personal assistance. Because LLMs can flexibly interpret new observations and decide subsequent actions in natural language, an LLM-centric agent architecture has become a natural default: after each observation, control is returned to the LLM, which decides the next step. This design preserves the generality and flexibility of LLMs, but it also makes the core control logic of the agent program highly dependent on LLM calls.

This convenience is not free. Each decision step introduces an expensive LLM call with growing context. The high cost of this architecture has motivated many efforts to accelerate LLM agents, such as batching, scheduling, and KV-cache management [16, 57, 62]. These techniques are effective, but most still optimize within the LLM-centric loop. They make model-mediated steps cheaper and faster, while the agent often still returns to the LLM after each observation, even when much of the execution is already structurally determined. As a result, such progress remains coupled to repeated LLM inference over growing context.

(a) LLM-Centric Execution: every tool result returns to the LLM



(b) Structure-Semantic Separation: runtime owns structural progress

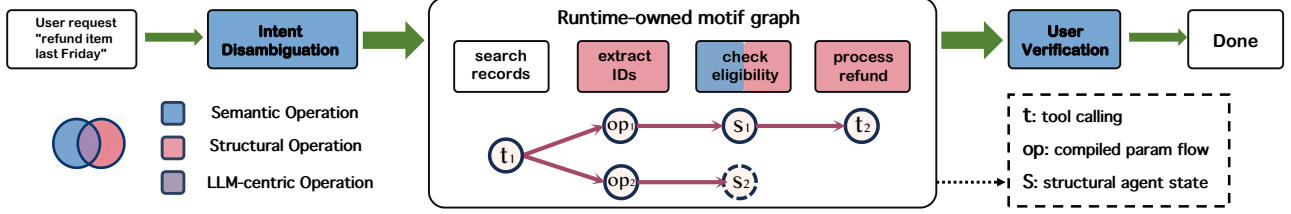


Figure 1. Running example of a refund task. (a) LLM-centric execution returns control to the LLM after every tool result. (b) Structure-semantic separation leaves semantic decisions to the LLM while executing stable tool order and parameter flow as a motif graph. Mixed colors denote evidence-dependent execution modes.

We argue that this bottleneck is architectural rather than merely computational. In many agent executions, especially after the initial ambiguity has been resolved, the next step is not always an open-ended semantic decision. It may follow a stable tool order, transfer a value from one tool result to a later input, release a downstream operation after its dependency is satisfied, or check a known precondition. Under an LLM-centric architecture, these structural transitions remain encoded in prompt text rather than executable runtime state. The runtime observes tool calls and tool results, but it does not own the recurring structure that connects them.

Tool use makes this opportunity especially clear. Tool calls expose explicit runtime boundaries, including tool names, input schemas, returned fields, and parameter dependencies across calls. These signals make the recurring execution structure observable to the runtime, unlike consecutive model-only reasoning steps whose boundaries depend heavily on prompting choices and model behavior.

The missing piece is a **redesign of control ownership**: the runtime should advance structural progress when current evidence is sufficient, while the LLM should be reserved for semantic gaps that structure cannot resolve.

Figure 1 illustrates this boundary with a refund task. In LLM-centric execution (a), the agent returns to the model after each tool result to choose the next step. Under structure-semantic separation (b), “the item I bought last Friday” still requires semantic mediation because the target is ambiguous. Once the relevant record is identified, the runtime can follow a motif graph to extract the order identifier, check eligibility, and process the refund accordingly. Each transition proceeds only when the required values and preconditions are verified.

If required evidence is missing, execution returns to the LLM as a semantic gap. The model receives only the current motif state and the evidence needed to resolve that

gap, rather than the full execution history. This shows that the boundary between structural execution and semantic reasoning is determined dynamically at runtime: verifiable transitions are executed structurally, while unresolved ones are delegated to the LLM. We call this boundary structure-semantic separation.

We build MotifAgent, an **agent-runtime and serving co-design system** that realizes structure-semantic separation. It represents recurring tool-use structure as motif graphs. *Positive motifs* encode local tool operations, parameter flow, dependencies, and the evidence required for each transition. The Structure Engine instantiates a matched motif as runtime state and advances only when the required values and preconditions are verified. At a semantic gap, the Semantic Engine receives a typed mediation request and invokes the LLM for bounded decisions such as value completion, candidate selection or clarification. *Negative motifs* are extracted from failed trajectories. They support early stopping on known failure patterns and provide corrective guidance during ReAct fallback. Neither motif type assumes a fixed whole-task workflow. The runtime advances verifiable progress and delegates unresolved decisions to the LLM.

Runtime-owned motifs also change what the serving system can observe. A conventional backend sees arrived LLM calls and token-level features, but not the dynamic local tool workflow or downstream operations behind each call. MotifAgent exposes motif stage, structured prompt prefix, and possible downstream releases. These serving-visible signals guide structure-aware scheduling and fine-grained KV-cache retention and eviction.

We evaluate structure-semantic separation on APiBank [19], TauBench [55], and AppWorld [41]. Compared with LLM-centric execution, MotifAgent reduces LLM calls by up to

27.1% and token consumption by up to 47.9%, while maintaining competitive task performance across the three benchmarks. The serving backend also uses motif progress to guide scheduling and retain prefixes likely to be reused after tool gaps.

Our contributions are summarized as follows:

- We identify the LLM-centric agent design as an architectural bottleneck and propose structure–semantic separation as a new execution boundary. It moves verifiable structural progress to the runtime and reserves semantic judgment for the LLM.
- We build MotifAgent, an agent-runtime and serving co-design system. Its motif graphs capture recurring tool-use structure, advance evidence-supported transitions, and scope LLM calls at semantic gaps.
- We design a serving interface and backend mechanisms that expose motif stages, structured prompt prefixes, and downstream-release hints. These signals guide structure-aware scheduling and fine-grained KV-cache retention and eviction.
- We implement MotifAgent on an SGLang-based serving stack. Across three agent benchmarks, it reduces LLM calls and token use at comparable task performance. Controlled replays evaluate latency and KV-cache effects under pressure.

2 Background

2.1 LLM Agent Control Loops

LLM agents interleave model decisions with observations from users, tools, and the environment [31, 45, 49, 56]. In a ReAct-style loop [56], the model selects an action, observes the result, and then decides the next step. This lets the agent respond to unexpected observations without a predefined workflow. Tool calls expose explicit runtime structure, such as typed inputs and outputs and parameter dependencies across operations. Their results can be passed directly to later operations, making tool order and parameter flow observable to the runtime. However, in an LLM-centric loop, this structure often remains embedded in the growing prompt history. When the same pattern recurs, the model is asked to interpret it again instead of the runtime executing it as state.

Several agent and language-model programming systems try to reduce the cost of repeated model calls through tool-use training, reusable prompts, declarative pipelines, structured generation, or workflow-style execution [4, 8, 14, 28, 38, 46, 61, 62]. These systems improve how LLM calls are organized or executed, but the default control loop often remains model-mediated. In this paper, we focus on a complementary boundary: when execution progress is already verifiable from runtime evidence, the runtime should advance it directly rather than asking the LLM to rediscover the same control structure.

2.2 LLM Serving and Prefix Caching

LLM serving systems batch requests to improve accelerator utilization. For decoder-only models, prefill processes the prompt and constructs KV states, while decode generates output tokens autoregressively. Serving systems such as ORCA and SARATHI improve utilization through iteration-level scheduling and chunked prefills [2, 57], while vLLM and others improve memory efficiency through PagedAttention and block-level KV-cache management [16, 27, 34]. Other recent architectures further improve serving efficiency by disaggregating prefill and decode [26, 63].

Prefix reuse is especially important for multi-call LLM applications [8, 35, 54]. Caching KV states for shared prompt prefixes avoids recomputing the shared part of prefill. SGLang [62] targets structured language-model programs and introduces RadixAttention, which stores reusable prefixes in a radix tree and reuses cached KV states across calls. Its scheduler can use radix-aware policies to select waiting requests based on token-prefix locality. Longest-prefix matching favors requests whose token prefix is already well represented in the cache, while DFS-style traversal favors dense prefix subtrees to improve locality.

However, agent requests arrive sequentially, separated by tool execution delays known as tool gaps [1, 18]. An LRU-based cache can evict a prompt prefix during a long tool gap, causing a cache miss when the tool returns.

Furthermore, these policies operate on tokenized prompt prefixes. Recent application-aware systems expose workflow information to the serving layer [21, 43], but such abstractions do not directly capture the observation-driven execution state of agentic control loops. Runtime semantic gaps, dynamic tool choices, and fallbacks can change which downstream operations become relevant.

Consequently, token-level cache identity and static workflow structure do not fully expose an agent’s execution stage, dependency frontier, or unresolved semantic gap. The serving backend therefore cannot directly reason about whether serving one LLM call will unblock downstream agent work, or whether an inactive prefix will become useful again after a tool gap. This semantic wall between the dynamic agent runtime and the serving backend motivates our cross-layer co-design.

3 Motivation

Structure–semantic separation shifts verifiable tool-use steps to the runtime and exposes execution state to the serving backend. The runtime executes recurring tool-use structure when current evidence supports the next transition (§3.1). It delegates unresolved decisions to the LLM through bounded semantic requests (§3.2). The backend uses the exposed progress to guide scheduling and cache retention (§3.3).

3.1 Observation 1: Tool-Use Traces Expose Executable Structure

ReAct-style agents are flexible because control can return to the model after every observation. This flexibility, however, also makes the model responsible for routine execution decisions. In recurring tool-use executions, successful traces often follow the same operation order and parameter-flow patterns, check the same preconditions, and enable downstream actions when their dependencies are satisfied. An LLM-centric control loop still incurs LLM calls and token costs to reconstruct this recurring structure during each execution.

This suggests that repeated tool-use behavior should be captured as executable runtime structure rather than prompt history. Prompt-only skills describe a workflow but leave its execution to the model. Predefined workflows handle deviations through explicitly encoded branches and recovery rules. A motif captures recurring structure from traces and combines tool order, parameter flow, guards, and dependencies with runtime evidence checks and semantic mediation, supporting partial execution and recovery.

Challenge 1: Make structure executable without freezing execution. A motif must let the runtime execute verified tool order, parameter flow, guard checks, and dependency release without fixing the task to one path. Its ready frontier and bound values must support partial execution, re-entry, and recovery when observations deviate from the common trace.

3.2 Observation 2: Structure Cannot Replace Semantics, But Can Bound It

Structure alone does not resolve every decision in agent execution. User requests may omit required values, refer to entities indirectly, or match multiple candidate motifs. Tool results may fail to satisfy expected guards. In these cases, the runtime may lack the evidence needed to determine tool arguments or validate the next transition. The question is therefore where to place the boundary between runtime execution and semantic reasoning.

MotifAgent draws this boundary through two runtime operations. A *structural operation* runs when current evidence is sufficient: it may invoke a known tool node, copy a verified value, check a guard, or release a downstream dependency. A *semantic operation* runs when evidence is insufficient: it asks the LLM to resolve a typed local gap, such as a missing slot, ambiguous entity, unexpected observation, or failed guard. During semantic mediation, the LLM answers a bounded question using motif state and compact evidence, while the runtime retains execution state.

Figure 2 compares input tokens per call across APIBank [19], TauBench [55], and AppWorld [41]. Semantic operations use fewer input tokens than ReAct-style calls by supplying compact motif state and relevant local evidence instead of the

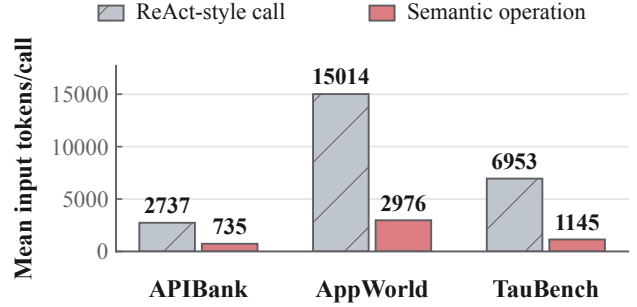


Figure 2. Semantic operations reduce input tokens by using compact motif state and local evidence.

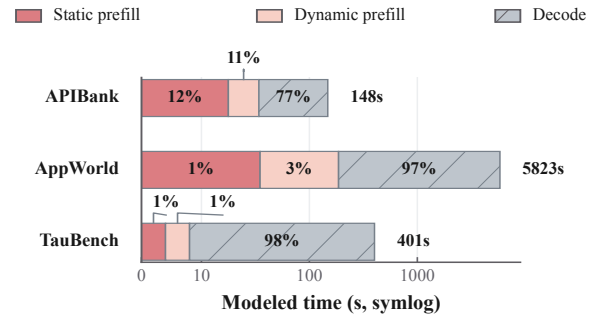


Figure 3. Decode accounts for the largest share of estimated backend time under motif-style execution, using token counts and fixed prefill and decode throughput assumptions.

full execution history. This comparison illustrates the input-token savings from narrowing the context supplied to each semantic operation.

Challenge 2: Bound semantic mediation while preserving runtime state. When structure is insufficient, the LLM must address the local gap while the runtime retains execution state. A typed mediation request identifies the missing slot, ambiguous candidate, or failed guard. The LLM answers the request, and the runtime passes the response back to the paused motif instance.

3.3 Observation 3: Bounded Semantic Operations Create a New Serving Workload

Structure-semantic separation changes both the context supplied to model calls and the execution information available to the backend. A short call may unblock downstream motif stages, while a fallback call may require more model work and delay task completion. A stable prefix may have no matching request in the current queue, yet a later LLM call may reuse it after a tool returns. Exposing motif progress makes these dependencies and reuse opportunities available to the backend as signals for scheduling and cache retention.

Figure 3 estimates backend work from token counts and fixed prefill and decode rates. In this timing model, decode

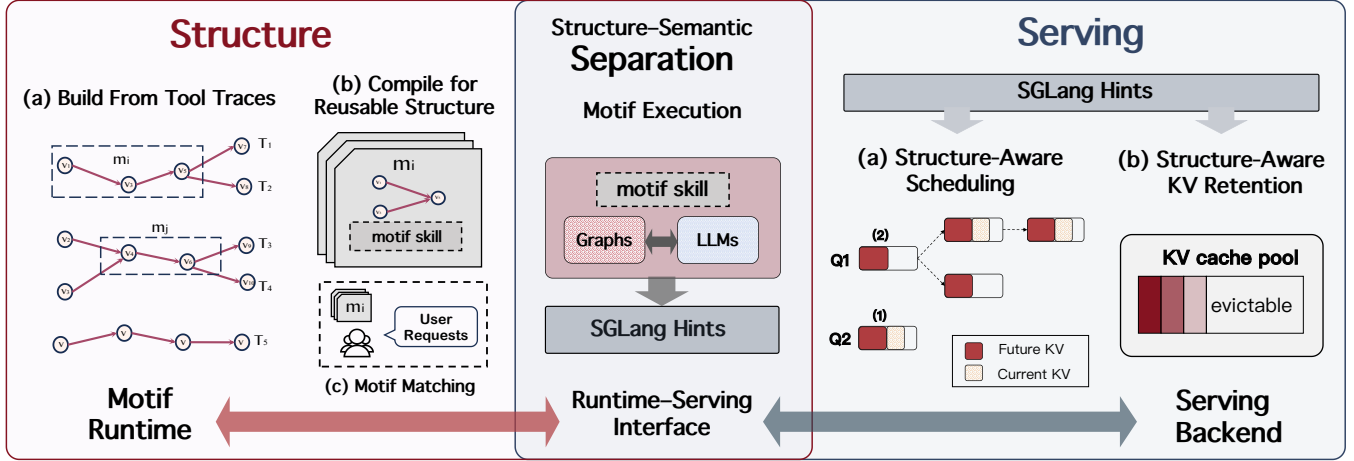


Figure 4. MotifAgent’s system architecture

remains the largest component after prompts shrink. The figure is not a measurement of GPU latency or queueing. It motivates a separate serving question: under queueing and cache pressure, can motif progress help the backend schedule requests and retain reusable prefixes?

To exploit motif information for serving, one option is to group requests by motif stage before backend admission. Such grouping may improve locality when similar stages arrive close together, but waiting for a group can delay admission when arrivals are dispersed. The runtime knows motif progress and possible downstream calls, while the backend has direct access to the waiting queue, radix-cache state, and running batch state. Exploiting structural information for serving therefore calls for runtime–backend co-design: the runtime supplies advisory metadata, and the backend combines it with current queue and cache state to guide scheduling and retention.

Challenge 3: Balance downstream reuse against current locality and cache pressure. Prioritizing requests for predicted downstream reuse may reduce prefix locality within the current batch, while retaining their prefixes may displace other useful cached context. The challenge is to weigh estimated downstream reuse benefits against immediate locality loss when scheduling, and decide which cached prefix ranges to prioritize for retention when cache space is limited. Both decisions must combine motif-provided reuse hints with the backend’s current queue and cache state.

These observations motivate three design requirements: evidence-supported structural execution, bounded semantic mediation, and serving policies that balance downstream reuse with current locality and cache pressure. The next section describes how MotifAgent realizes these requirements.

4 MotifAgent Design

This section describes how MotifAgent realizes structure–semantic separation. We first define the execution boundary (§4.1) and executable motifs (§4.2), then describe structure-aware scheduling (§4.3) and KV-cache retention (§4.4).

4.1 Overview

MotifAgent executes recurring tool-use steps outside the default LLM control loop. Structural execution advances only when current evidence supports the next transition. When evidence is insufficient, the runtime invokes typed mediation or fallback to resolve the remaining uncertainty. The runtime also exposes execution progress to the serving backend to guide request scheduling and cache retention.

Architecture. Figure 4 shows three layers: the agent runtime, the system interface, and the serving backend. An offline compiler produces motif libraries for the agent layer. At runtime, a matched positive motif becomes an instance that tracks execution progress, completed steps, current bindings, and unresolved requirements. Support traces provide evidence for compiled parameter-flow rules. The Structure Engine advances ready nodes after checking their arguments and preconditions. The Semantic Engine resolves local uncertainty through typed mediation. The interface carries advisory `sglang_hints` to the backend for scheduling and retention.

Execution bound. We model a tool-use execution as structural transitions interleaved with semantic decisions. A structural transition is runtime-verifiable: it may invoke a tool with determined arguments, copy a value from a tool result, check a precondition, update state, or enable downstream nodes once their dependencies are satisfied. A semantic decision requires model reasoning when the available evidence is insufficient, for example, to resolve a missing

value, disambiguate an entity, select a candidate, or interpret an unexpected observation.

For a fixed execution path of task τ containing at least one required model-resolved decision, let $\mathcal{D}_\tau = (V, E)$ be its semantic dependency DAG, with repeated decisions unrolled into distinct nodes. Each node represents a required semantic decision. An edge $(u, v) \in E$ indicates that resolving v requires evidence that becomes available only after u and cannot be obtained within the same model invocation. Let $\text{Paths}(\mathcal{D}_\tau)$ denote the directed paths in this graph. The semantic critical path counts decision nodes on the longest path:

$$\text{SCP}(\tau) = \max_{\pi \in \text{Paths}(\mathcal{D}_\tau)} |\pi|. \quad (1)$$

Any execution that resolves these decisions while respecting their evidence dependencies requires at least $\text{SCP}(\tau)$ sequential LLM rounds. If a step-by-step baseline follows the same execution path using $N_{\text{step}}(\tau)$ sequential model-mediated rounds, its reduction factor is bounded by:

$$\text{RoundReduction}_{\max}(\tau) \leq \frac{N_{\text{step}}(\tau)}{\text{SCP}(\tau)}. \quad (2)$$

This conditional bound concerns the reduction in sequential LLM rounds along a fixed execution path. Motifs reduce model involvement in structural transitions, but unresolved runtime uncertainty may require additional mediation or fallback rounds beyond $\text{SCP}(\tau)$ even when motif coverage is high.

4.2 Motif Representation and Compilation

MotifAgent uses two kinds of motifs. *Positive motifs* encode recurring successful tool-use behavior as executable operation graphs. *Negative motifs* encode repeated terminal failure patterns. They act as guards during ReAct fallback and are never executed as plans.

Algorithm 1 summarizes offline compilation. Deterministic stages mine recurring structure, bind support evidence, and construct motif skeletons and connectors. `LLMCOMPILE` proposes semantic and execution metadata, including parameter-flow rules, tool effects, and parallelism annotations. `VALIDATEPAIR` checks these candidates against tool schemas and support evidence, applies rule-based filtering or demotion, and may request a limited number of LLM revisions using validation feedback. Failed or abandoned trajectories contribute negative guards rather than executable plans.

Motif maintenance. Tool documentation or schema changes require revalidation of affected motifs. Repeated execution failures also warrant checking whether their assumptions remain valid. Motifs with outdated rules require recompilation using updated specifications and compatible support traces,

Algorithm 1 Motif compilation

Require: Trajectories $\mathcal{T}$, tool schemas Σ
Ensure: Positive motifs $\mathcal{P}$, negative guards $\mathcal{N}$, connector table C

- 1: $\hat{\mathcal{T}} \leftarrow \text{NORMALIZE}(\mathcal{T}, \Sigma)$
- 2: $(\mathcal{T}^+, \mathcal{T}^-) \leftarrow \text{SPLITBYOUTCOME}(\hat{\mathcal{T}})$
- 3: $S \leftarrow \text{MINESIGNATURES}(\mathcal{T}^+)$
- 4: $C \leftarrow \text{BUILDCONNECTORS}(S, \hat{\mathcal{T}})$
- 5: $\mathcal{P}, \mathcal{N} \leftarrow \emptyset, \emptyset$
- 6: **for all** $s \in S$ **do**
- 7: $B \leftarrow \text{BUILDSKELETON}(s, \Sigma, C)$
- 8: $E \leftarrow \text{BINDSUPPORT}(s, \mathcal{T}^+)$
- 9: $A \leftarrow \text{LLMCOMPILE}(B, E)$
- 10: $A \leftarrow \text{VALIDATEPAIR}(A, B, E, \Sigma)$
- 11: $\mathcal{P} \leftarrow \mathcal{P} \cup \{A\}$
- 12: **end for**
- 13: $\mathcal{N} \leftarrow \text{MINEFAILUREGUARDS}(\mathcal{T}^-)$
- 14: **return** $(\mathcal{P}, \mathcal{N}, C)$

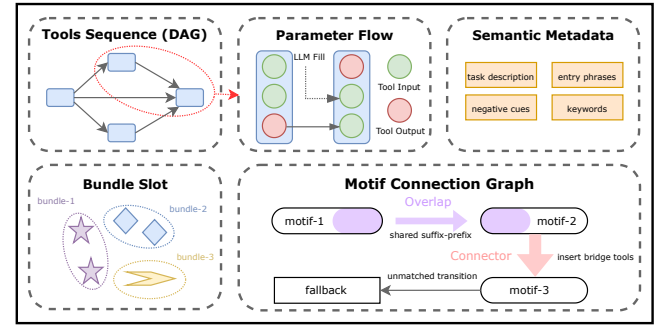


Figure 5. An executable motif combines graph structure with semantic and transition metadata.

separately from fallback during the current task. Silent semantic changes that preserve existing interfaces may evade runtime checks and require external revalidation.

Figure 5 separates executable structure from supporting metadata. The operation graph contains tool nodes, dependencies, and node types for tool calls, verified extraction, bounded LLM filling, response generation, and mediation handoff. Parameter-flow edges identify inputs copied from prior outputs, extracted from local evidence, or left as semantic requirements.

The dialogue contract bounds uncertainty within a motif, while connection hints describe continuation across motifs. The contract limits which missing values may be requested from the user or LLM, and bundleable slots let related values be resolved together. Connection hints identify possible continuations through suffix-prefix overlap or observed connector tools. They help re-entry and serving lookahead, but never override runtime evidence checks.

At runtime, the agent matches a positive motif against the current request and environment state and instantiates its operation graph. The Structure Engine identifies ready nodes, grounds their arguments in current evidence, and checks their guards before invoking tools. Tool results update the instance state and satisfy dependencies for subsequent nodes. For unresolved arguments or local interpretation decisions, the runtime constructs typed mediation requests that specify the value or decision the LLM must resolve and provide compact motif state and relevant local evidence. If no executable motif matches, or the required reasoning exceeds the motif’s local scope, execution returns to the agent’s general LLM control loop.

Unlike prompt-only skills, executable motifs let the runtime propagate values, check preconditions, and invoke structurally determined tools directly. The runtime can also continue across motifs. For example, connection hints may identify a verification motif as a continuation of a retrieval motif. Before starting verification, the runtime checks whether the retrieved results supply the required inputs and satisfy the next motif’s entry conditions.

Suspension and re-entry. The runtime suspends a motif instance if required arguments cannot be resolved, preserving its execution position, completed steps, tool results, and bindings. Mediation or fallback can provide new evidence for re-entry. Before continuing, the runtime invalidates bindings affected by changed evidence, rechecks the required arguments and guards, and resumes from the next unfinished step.

4.3 Structure-Aware Scheduling

Structure-semantic execution makes each LLM call more than a token sequence. Besides its prompt, the call carries local workflow context: its motif id, stage id, tool-schema prefix, stable prompt layout, and the downstream calls that may become ready after it finishes. We call this policy Structure-Aware Scheduling (SAS). It uses motif context as a conservative lookahead layer on top of radix scheduling, whose conventional policies observe only current token-prefix locality and cache state.

For each remaining LLM call, MotifAgent attaches advisory `sglang_hints` to the OpenAI-compatible request. These hints describe the current motif and stage, canonical tool-schema prefixes, a stable prefix ladder, and possible downstream calls. Predicted downstream prefixes guide scheduling, while cache-range priorities and expiration times guide temporary retention.

Scheduling balances estimated downstream reuse against current prefix locality. The backend matches predicted downstream prefixes against prefix metadata for other requests in the waiting queue. A match indicates that completing the current call may enable downstream calls that share prefixes with queued requests. The backend estimates this reuse benefit based on shared prefix length and the number of matching

Prompt layout for motif-based LLM requests

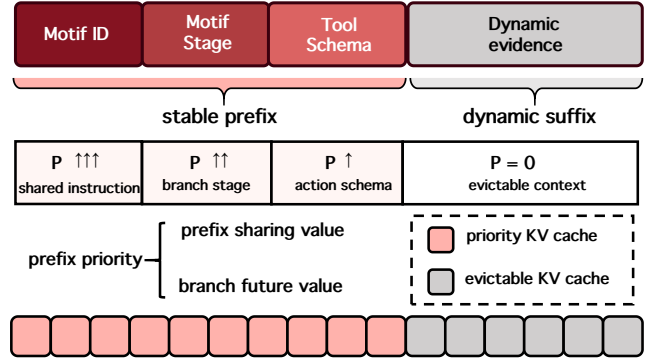


Figure 6. Fine-grained KV-cache retention protects reusable motif prefixes while dynamic evidence remains evictable.

queued requests. However, promoting the current request may favor a sparse radix subtree over a denser sibling subtree, disrupting current locality. To estimate this cost, the backend traverses the request’s radix-tree path, finds the largest positive difference between the queued-request count of a competing sibling subtree and that of the promoted subtree, and scales this difference into a locality-cost estimate.

The loss-aware policy discounts the estimated reuse benefit by the weighted locality cost and converts accepted gains into bounded bonuses to radix-subtree scores. These scores adjust the DFS ordering of the waiting queue. The backend admits requests in the resulting order subject to its existing memory and batch-size limits. With speculative release credit disabled, predictions without a match to another queued request contribute no bonus. If no positive bonus is accepted, scheduling follows the base DFS policy.

4.4 Fine-Grained KV-Cache Retention

Motif execution also changes cache management. During multi-step agent execution, an LRU-based policy may evict prefixes that will soon be revisited after a tool gap. In contrast, request-level KV-cache pinning may preserve the entire request prefix, including dynamic evidence that is unlikely to be reused. This coarse policy increases cache pressure. Motifs provide a finer-grained signal: they predict which stable prefixes are likely to be used by upcoming downstream calls and assign higher retention priority only to those ranges.

Figure 6 illustrates selective retention with a layered prompt layout. Stable motif context and tool schemas precede task-specific inputs and tool observations. Retention hints target prefix ranges expected to be reused after tool gaps, avoiding the cache-space cost of retaining the entire prompt.

Retention priorities reflect estimated reuse value per occupied KV block, with layer-dependent weights favoring upstream prefixes. The backend associates radix-tree nodes

with retention priorities and expiration times. When a retention range covers only part of a node, it splits that node at the range boundaries and assigns priority to the covered tokens. Under cache pressure, lower-priority ranges are evicted first, while higher-priority ranges remain eligible for eviction when memory must be reclaimed. Expired retention hints no longer contribute priority.

5 Implementation

MotifAgent is implemented as a Python agent runtime integrated with a modified SGLang v0.5.9 backend. The implementation consists of an agent layer, a system interface layer, and a serving layer.

Agent layer. The runtime compiles tool-use traces into motif libraries and matches motifs against the current request and environment state. Each motif invocation maintains a MotifFrame containing the execution position, completed tools, tool results, unresolved parameters, and validated semantic bindings. Frames are preserved across suspension and fallback, while evidence dependencies are tracked to invalidate stale bindings before reuse. Typed semantic responses are resolved and validated before execution. During fallback, the runtime matches tool sequences and error signatures against negative motifs to identify known failure patterns. Matches can trigger early termination or provide recovery guidance to the LLM.

System interface layer. The runtime serializes structural metadata into `sglang_hints`, carried through the OpenAI SDK’s `extra_body` field. The backend parses these fields into `StructuredRequestHints` and propagates them with internal generation requests. The metadata includes motif and stage identifiers, prefix ladders, downstream-release estimates, cache-retention ranges, priorities, and expiration times. Retention offsets are clamped to the cached token length and rounded down to cache-page boundaries.

Serving layer. The modified scheduler implements SAS using information including motif id, motif stage and downstream releases. It adds bounded motif-aware adjustments to DFS subtree scores and reorders the waiting queue. Existing admission logic then selects requests under memory and batch-size constraints. The radix cache splits nodes at normalized retention boundaries and assigns priorities and expiration times to the corresponding nodes. Under cache pressure, eviction favors lower-priority KV blocks.

6 Evaluation

We evaluate the two effects of structure–semantic separation. First, three tool-use benchmarks test whether runtime-owned motifs reduce LLM calls and tokens while preserving task outcomes. Then controlled request replays test whether serving-visible motif progress affects latency and KV-cache retention under pressure.

6.1 Runtime-Layer Setup

Setup. The experimental host has one NVIDIA A100 GPU with 80 GB of memory and dual Intel Xeon Gold 6348 CPUs. Unless otherwise stated, we use DeepSeek-V4-Flash for all LLM calls in runtime-layer experiments. We use the locally deployed `sentence-transformers/all-MiniLM-L6-v2` encoder with cosine similarity for motif matching.

Workloads. We evaluate on APIBank [19], TauBench [55], and AppWorld [41], covering single-turn tool use, multi-turn tool use, and API-based application automation, respectively. We report APIBank accuracy, TauBench task success rate, and AppWorld mean task pass percentage, together with LLM calls and prompt-plus-completion tokens.

Data splits. We separate compilation and evaluation tasks by task ID and freeze the compiled motif library before evaluation. APIBank uses three splits of its 50 Level-3 task IDs into disjoint compilation and evaluation pools. TauBench uses three splits, each with 58 compilation and 57 evaluation task IDs. AppWorld uses 90 training tasks for compilation and 100 `test_normal` tasks for evaluation, with three evaluation repetitions. Each split or repetition uses 200 compilation inputs and 200 evaluation episodes, allocated as evenly as possible across task IDs within their respective pools.

Baselines. We compare MotifAgent with ReAct [56], ExpeL [61], AWM [46], Agent-KB [39], and APC [59]. ReAct provides an LLM-centric baseline, while the other methods reuse experience or memory. All methods use the same evaluation episodes within each split or repetition. Methods requiring experience inputs receive the same source trajectories from the compilation pool.

6.2 Runtime-Layer Results

We study whether structure–semantic separation can reduce repeated model-control work across different tool-use workloads while maintaining competitive task performance.

Table 1 shows that MotifAgent reduces token usage and LLM calls relative to every evaluated baseline across all three benchmarks, while maintaining competitive task performance. On AppWorld, for example, MotifAgent achieves a 66.3% pass rate with 130.0k tokens and 12.93 LLM calls per task. Success rates vary across workloads, and MotifAgent does not consistently achieve the highest accuracy or pass rate.

These results support the runtime-layer goal of reducing repeated model-mediated control: the runtime executes verified tool-use transitions, while the LLM remains available for semantic gaps. To isolate the contribution of runtime execution, we next compare MotifAgent with a motif-as-prompt control that supplies the same motif structure as textual guidance.

6.2.1 Motif as Prompt. Existing experience-enhanced and skill-based agents often reuse past behavior as textual guidance. MotifAgent introduces structural experience by

Table 1. Runtime-Layer results across three tool-use benchmarks. $\uparrow$ indicates higher is better and $\downarrow$ indicates lower is better. Best results are shown in **bold** and second-best results are underlined.

Benchmark	Metric	ReAct	ExpeL	AWM	Agent-KB	APC	Motif (ours)
APIBank	Accuracy (%) $\uparrow$	56.33	72.00	67.67	67.00	78.50	<u>76.83</u>
	Avg. Tokens (k) $\downarrow$	9.63	16.09	11.68	11.53	<u>6.42</u>	5.02
	Avg. LLM Calls $\downarrow$	<u>3.61</u>	3.64	3.73	3.66	4.72	3.38
TauBench	Task success rate (%) $\uparrow$	62.50	<u>74.00</u>	78.00	72.00	69.00	<u>74.00</u>
	Avg. Tokens (k) $\downarrow$	94.91	100.67	104.87	98.93	<u>90.49</u>	78.80
	Avg. LLM Calls $\downarrow$	14.84	<u>14.00</u>	14.34	14.51	15.74	11.97
AppWorld	Pass Percentage (%) $\uparrow$	67.48	69.88	62.81	<u>67.87</u>	56.37	66.26
	Avg. Tokens (k) $\downarrow$	204.17	213.67	205.77	245.84	<u>193.37</u>	130.04
	Avg. LLM Calls $\downarrow$	17.74	17.51	17.68	<u>17.40</u>	19.42	12.93

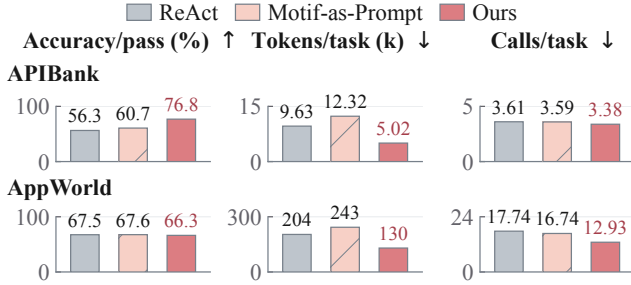


Figure 7. Motif-as-prompt ablation on APIBank and AppWorld. Motif-as-prompt uses the same compiled motif library and matcher as MotifAgent, but only renders the matched motif as a prompt hint for a ReAct agent.

compiling recurring tool-use graphs into executable motifs to let the runtime track parameter dependencies and execute structurally determined tool calls directly.

To test whether executable motif skills provide benefits beyond textual guidance, we compare MotifAgent with a motif-as-prompt variant. It uses the same motif library and matcher, but adds the matched motif as a prompt hint and leaves control with the ReAct agent.

Figure 7 shows that the prompt hint alone does not reproduce the runtime benefit. On APIBank, motif-as-prompt improves accuracy from 56.3% to 60.7%, but increases average tokens from 9.63k to 12.32k. In contrast, MotifAgent reaches 76.8% accuracy while reducing tokens to 5.02k and LLM calls to 3.38.

On AppWorld, motif-as-prompt has almost the same pass rate as ReAct, 67.6% versus 67.5%, but increases average tokens from 204k to 243k. MotifAgent achieves a comparable pass rate of 66.3%, while reducing average tokens to 130k and LLM calls from 17.74 to 12.93.

These comparisons support the architectural claim: describing a motif in a prompt does not reproduce the lower

Table 2. Model-robustness diagnostics on APIBank. Accuracy measures ordered API coverage over all 100 episodes. Tokens and calls are episode averages, including final responses and excluding compilation.

(a) Execution models; compiled with Plus				
Exec.	Method	Acc. (%)	Tok. (k)	Calls
Plus	ReAct	61.0	9.85	3.54
	ExpeL	76.0	16.68	3.68
	MotifAgent	72.0	4.43	3.08
9B	ReAct	64.0	9.58	3.48
	ExpeL	72.0	16.34	3.61
	MotifAgent	72.0	4.45	3.04
(b) Compilation models; executed with Plus				
Compiler		Acc. (%)	Tok. (k)	Calls
Plus		72.0	4.43	3.08
9B		64.0	4.27	2.90

model-control cost of executing it in the runtime. The Structure Engine advances verified tool and parameter-flow transitions, while the LLM handles the remaining semantic gaps.

6.2.2 Model Robustness. We evaluate Qwen3.7-Plus (Plus) and Qwen3.5-9B (9B) on the same 100 APIBank episodes (25 held-out task IDs, four executions each), disjoint from compilation IDs. Both motif compilers use the same 200 training trajectories; ExpeL knowledge is frozen and train-only. Reusing Plus-to-Plus yields seven configurations and 700 episodes.

As Table 2 shows, MotifAgent reduces tokens by 72.8–73.4% and LLM calls by 15.8–16.3% relative to ExpeL. Its accuracy exceeds ReAct with both executors, matches ExpeL with 9B, and is lower with Plus. With Plus execution fixed, the 9B-compiled library yields 64.0% accuracy versus 72.0%

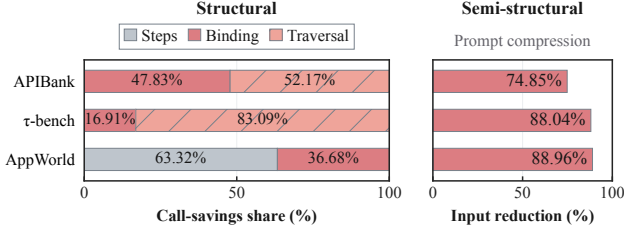


Figure 8. Sources of runtime savings. Left: each mechanism’s share of recovered structural call savings. Right: input-token reduction relative to ReAct for matched local resolution operations. AppWorld’s prompt reduction uses approximate same-task/tool matching over five execution episodes, including selection overhead.



Figure 9. AppWorld results by task difficulty. Harder tasks reduce pass rate and increase model-control work for all methods, but MotifAgent stays on a lower-token and lower-call execution path.

for Plus compilation. These observations are conditional on the selected split and frozen libraries.

6.3 Breakdown Experiments

Runtime Efficiency Gain Breakdown. Figure 8 separates two sources of runtime savings. For **structural operations**, MotifAgent avoids LLM decisions through fixed-step execution, argument binding from runtime state, and traversal over collections. Among the recovered operations, traversal dominates τ -bench, binding and traversal contribute almost equally on APIBank, and fixed-step execution contributes most on AppWorld. For **semi-structural operations**, the LLM still resolves semantic choices, but receives a local prompt focused on the relevant context and missing arguments. The right panel shows the resulting input-token reduction: these operations save prompt-processing work while retaining the semantic decision.

Task Difficulty Breakdown. This experiment studies how MotifAgent behaves as task difficulty increases. We group AppWorld tasks into Level 1, Level 2, and Level 3, and report pass rate, average tokens per task, and average LLM calls per task.

Figure 9 shows the expected difficulty trend. As tasks become harder, pass rate drops and both token usage and LLM

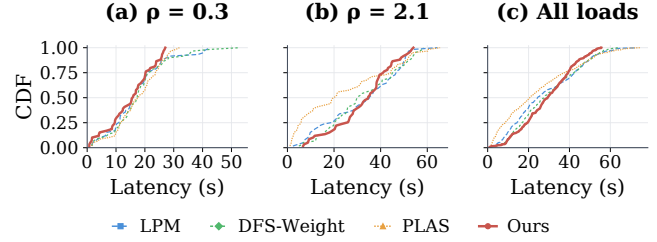


Figure 10. Per-request latency distributions in the AppWorld replay

calls increase. MotifAgent follows the same trend, but it uses fewer tokens and fewer LLM calls than the baselines across difficulty levels. This suggests that motifs do not remove all semantic work, but they reduce repeated structural control even when tasks become harder.

6.4 Serving-Layer Setup

Workload. We use APIBank-derived requests for the scheduling and KV-cache retention experiments. The requests retain the prompts and output lengths recorded during the runtime evaluation. The AppWorld latency ECDF uses an AppWorld request replay. Its plotted low- and high-load slices are labeled $\rho = 0.3$ and $\rho = 2.1$. Here, $\rho = \lambda/\mu$ denotes the offered load.

Replay backend. APIBank requests retain their recorded prompts and target output lengths. We replay the same content under every backend policy. Fixing output length controls decode work; request latency is measured anew in each run.

Serving platform and resource limits. The APIBank serving comparisons use Qwen3-14B on one A100 80 GB PCIe GPU and a modified SGLang backend. The scheduling runs allow 49,152 KV tokens and 16 concurrent requests, with retention disabled. The KV-retention runs allow 6,000 KV tokens and 8 concurrent requests, using LPM, a 2 s priority TTL, and a 30 s completed-prefix history window. Each run starts a fresh server with an empty cache.

Baselines. For our Structure-Aware Scheduling (SAS), we compare against three backend schedulers. LPM batches requests by the longest reusable prefix, and DFS-weight prioritizes heavily shared branches in the prefix tree [62]. They both use token-prefix locality, but do not use future request information. We also compare with the PLAS policy of Agentix (originally Autellix) [24], which prioritizes LLM calls from programs that have received less completed service. This tests whether finer-grained motif-stage signals provide benefits beyond prefix locality and program-level attained service. For fine-grained KV retention, we compare against LRU and request-level retention. Request-level retention assigns priority to the entire prefix, while Layered assigns

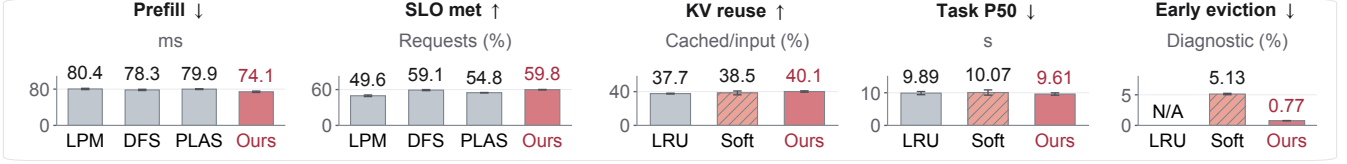


Figure 11. Scheduling and KV-cache retention on the APIBank request replay. Error bars show 95% Student-*t* confidence intervals over ten runs. Ours denotes SAS for scheduling and Layered for retention. Soft denotes request-level retention. Early eviction measures the fraction of protected KV blocks evicted before their retention period expires.

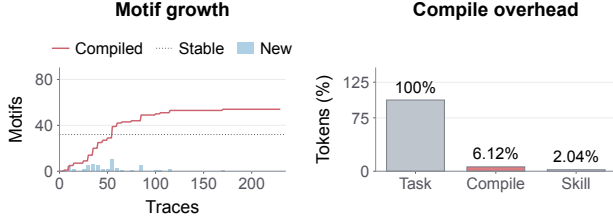


Figure 12. Motif compilation growth and token overhead on AppWorld

priority to selected prompt regions. Both permit eviction under memory pressure.

Metrics. We measure request latency from eligibility to completion, including client delay, queueing, prefill, and decoding. A request meets its SLO if it completes within the workload deadline. We also report mean per-request prefill-stage wall-clock latency and latency ECDFs. For KV retention, we report the first-admission cached/input token ratio, aggregated by token count, and each run’s task P50 from arrival to completion. Task performance is measured separately in the runtime-layer evaluation.

SLO calibration. For APIBank, we measure request P95 in a serial, low-pressure run and multiply it by two to set the deadline. The reported request thresholds are 5.72 s for APIBank and 120 s for AppWorld, fixed across policies within each workload. Calibration flushes the cache between tasks, whereas replay reuses cache within each initially cold run.

Load protocol. The APIBank scheduling and retention comparisons use the same Poisson arrival distribution and frozen trace for every policy and repetition within each experiment. Each trace contains 200 tasks and 671 requests, at 6 tasks/s for scheduling and 0.75 tasks/s for retention.

6.5 Serving-Layer Results

We first use APIBank to isolate how backend-visible structure affects serving behavior. It contains dense and regular motif reuse.

For scheduling, MotifAgent achieves the lowest mean per-request prefill latency and the highest average request-SLO attainment in Figure 11. LPM and DFS prioritize current prefix locality, while PLAS uses program-level attained service.

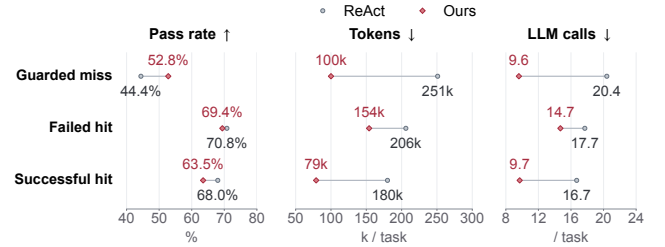


Figure 13. AppWorld runtime behavior by motif outcome. Guarded miss denotes unmatched positive motifs and matched negative motifs. Failed hit denotes partial motif execution with fallback. Successful hit denotes completed motif execution.

MotifAgent uses downstream-release hints to identify future prefixes. It credits their reuse only when they match requests already in the waiting queue.

The cache comparison tests whether motif hints protect prefixes across tool gaps. Request-level retention prioritizes whole static prefixes, while MotifAgent retains reusable prompt regions. Across the evaluated runs, Layered achieves the highest mean admission-time KV reuse and the lowest average task P50. Its early-eviction diagnostic is also markedly lower than that of request-level retention.

Figure 10 extends the latency comparison to AppWorld. In the plotted replay, the structure-informed policy does not always minimize median latency, but it has a shorter tail than the prefix-only and program-level baselines.

6.6 Overhead and Scalability Analysis

Compilation overhead. Figure 12 shows that motif compilation grows slowly as more successful AppWorld traces are observed. Under shuffled trace order, the final stable library contains 32 motif signatures, and MotifAgent submits 54 candidates to the LLM-assisted motif compiler. The last 50 traces introduce no new compile candidate. This shows that motif discovery becomes sparse after common structures have been covered.

The LLM compile cost is also small after reuse. Estimated from existing execution traces, one compile event produces three new motifs and uses 12.5k tokens in total. This is 6.12%

Table 3. Latency sensitivity to motif coverage

Coverage (%)	20	35	50	65
P50 reduction (%)	1.0 ± 2.5	4.4 ± 6.5	31.6 ± 3.0	42.6 ± 1.5

of ReAct’s average per-task token consumption on AppWorld. Thus, compilation is not free, but its LLM cost is paid only for newly discovered motif signatures and is amortized by later reuse.

Sensitivity to motif coverage. To examine sensitivity to motif coverage, we vary the fraction of motif requests in a controlled APIBank dependency replay with 100 requests per arm. Table 3 reports the P50 latency reduction relative to LPM, as mean $\pm$ sample SD over three paired seeds. Higher coverage exposes more downstream dependencies to the scheduler, creating additional opportunities for structure-aware scheduling. This experiment uses controlled successor releases and a workload distinct from the main scheduling replay.

6.7 Motif Behavior and Failure Analysis

To further explain where MotifAgent gains come from, we break down AppWorld executions by motif outcome. We consider three cases: no progress motif is matched and the runtime enters guarded fallback; a motif is matched but execution partially falls back to semantic operations; and a motif is matched and executed successfully.

Figure 13 shows that MotifAgent still helps when motifs are not fully executable. A common failure mode in agent execution is local error loops. During motif compilation, **repeated tail failures** can be turned into guarded fallback motifs, which stop unproductive execution when no progress motif is matched. In the motif-miss group, MotifAgent achieves a higher pass rate while using less than half the tokens and LLM calls of ReAct.

When a matched motif cannot complete the task on its own, MotifAgent still preserves task performance while reducing model work. Its pass rate remains close to ReAct, 69.4% versus 70.8%, and token usage drops from 206k to 154k. This shows that even **partial structural execution** is useful: the LLM still resolves semantic gaps, while the matched motif constrains what it needs to decide.

When motif execution succeeds, MotifAgent reduces token usage from 180k to 79k and LLM calls from 16.7 to 9.7. Its pass rate is slightly lower than ReAct because local motif completion does not imply that the full task succeeded.

We also audit a conservative negative-motif guard on 168 historical AppWorld ReAct trajectories. The guard requires a repeated match to a failure pattern learned from task-disjoint training traces. In a five-fold shadow counterfactual, it would have stopped 20 continuations; all 20 failed in the recorded

trajectories. No false stop is observed among those activations, although the Wilson 95% lower bound on precision is 83.9%. The trigger covers 20 of 145 failed trajectories (13.8%) and would avoid 333 recorded LLM calls. This offline test checks guard selectivity on saved traces. It does not measure online task outcomes after stopping or recovery.

7 Discussion and Limitations

7.1 Why This Is a Systems Problem

Structure–semantic separation changes who owns execution progress. The agent runtime must track graph state, verify evidence, and recover at semantic gaps. It can then expose motif progress to the serving backend, which schedules the remaining LLM calls and retains reusable prefixes under pressure. This cross-layer boundary is the systems contribution.

7.2 When MotifAgent Helps

MotifAgent is most likely to help when tool-use workflows recur, schemas and parameter flow remain stable, and runtime evidence can verify transitions. Its serving policies have additional opportunities under queueing and KV-cache pressure, especially when workflows differ in downstream value or reuse structural prefixes after tool gaps. Low motif coverage or changing tool semantics increases fallback; under low load or with short, non-reused prompts, serving gains should be smaller.

7.3 Limitations and Future Work

Our task-disjoint evaluations cover three benchmarks, but do not establish cross-benchmark transfer or robustness to silent schema drift. A motif hit verifies local progress, not task completion (§6.7). Negative guards may stop recoverable tasks, and their online false-positive rate remains unmeasured. Our compilation probe measures LLM tokens but excludes trajectory collection, validation, and maintenance. A concrete direction for agent self-improvement is to test whether repeated semantic decisions can become evidence-checked motif transitions without increasing failures under schema drift.

The isolated scheduling and KV-cache retention experiments show modest serving gains, whose magnitude depends on system load, cache pressure, and motif coverage. Our runtime architecture reduces input tokens but also changes how request contexts are organized, potentially reducing prefix reuse across turns within the same task. Consequently, when ReAct’s input prefixes are largely served from the KV cache, fewer input tokens do not necessarily translate into lower end-to-end latency.

8 Related Work

8.1 LLM Agents and Reusable Tool-Use Structure

LLM agents interpret observations and act through external tools [45, 49]. ReAct returns control to the model after each observation [56], HuggingGPT delegates tool selection to the LLM [32], and ReWOO decouples planning from observations [52]. Toolformer, ToolLLM, and API-Bank investigate API use [19, 30, 31], while WebArena, τ -bench, AppWorld, OSWorld, and SWE-agent benchmark long-horizon tool-use tasks [41, 51, 53, 55, 65].

Experience reuse and programmability offer complementary approaches. ExpEL, AWM, Agent-KB, and APC capture reusable insights, workflows, or plans [39, 46, 59, 61], while DSPy, SGLang, and LMQL structure model programs [4, 14, 62]. AutoGen, AgentScope, Voyager, and Language Agent Tree Search organize agent interactions or search [7, 44, 48, 64]. Chameleon supports tool composition [23], and TaskWeaver generates code for plugin execution [28]. However, descriptions and memories alone do not make recurring decisions executable by the runtime.

Beyond experience reuse, prior work structures agent computation through graph-based reasoning [3], parallel function calls [15], workflow search [58], and shared agent services [25]. Closer to our setting, AutoTool learns parameter-aware tool transitions [12], AutoFlow generates workflows [20], and GraSP executes verified skill graphs with local repair [50]. Unlike AutoTool, MotifAgent advances tool and parameter-flow transitions only with runtime evidence. While GraSP also executes verified graphs, MotifAgent exposes motif progress to the serving backend and handles semantic gaps through typed mediation.

8.2 LLM Serving and KV-Cache Management

LLM serving improves inference efficiency through batching, scheduling, and memory management. ORCA and SARATHI optimize iteration-level batching [2, 57], while FastServe, learning-to-rank scheduling, and VTC address request ordering and fairness [6, 33, 47]. vLLM manages KV-cache allocation, whereas SGLang and Preble exploit prefix reuse [16, 35, 62]. Llmunix supports request migration [36], while Splitwise and DistServe disaggregate prefill and decoding [26, 63]. These techniques optimize model execution, whereas MotifAgent also reduces model-mediated steps through executable tool-use structure.

Application-aware serving exploits richer signals beyond tokens, including Ayo’s primitive-level dataflow [37], Parrot’s semantic variables [21], and Pie’s serving-side handlers [9]. Agentix (formerly Autellix) schedules agent programs based on attained service [24]. MotifAgent instead exposes evidence-validated motif progress and potential downstream releases, allowing the backend to consider future agent execution alongside current request locality.

KV-cache management addresses memory pressure and redundant computation. vAttention and FlexGen improve KV storage [27, 34], while InfiniGen, CacheGen, Mooncake, and H2O explore prefetching, streaming, disaggregation, and selective retention [17, 22, 29, 60]. Prompt Cache and CacheBlend enable cross-prompt reuse [8, 54], whereas InferCept and Continuum address interruptions and multi-turn gaps [1, 18]. MotifAgent complements these techniques with motif-derived hints that protect reusable prompt regions across tool gaps.

8.3 Workflow Scheduling

Classical workflow scheduling exploits task dependencies for execution and resource allocation. Foundational work studies precedence constraints and critical-path scheduling [10, 13], while HEFT maps dependent tasks onto heterogeneous processors [40]. Tetrisched adapts to changing cluster demand [42], and Pegasus and GRAPHENE exploit workflow dependencies for data movement and cluster packing [5, 11]. These systems demonstrate the value of dependency information beyond the immediate ready queue.

LLM-agent serving introduces additional constraints. Requests share prompt prefixes and compete for KV-cache space, while tool observations and semantic gaps affect downstream readiness. Dependency information alone cannot determine request priority. MotifAgent combines motif progress with backend queue and cache state, balancing downstream reuse against current prefix locality and cache pressure.

9 Conclusion

MotifAgent addresses the LLM-centric architectural bottleneck by changing who controls routine execution. Executable motif graphs let the runtime advance evidence-supported tool transitions; typed mediation reserves the LLM for semantic gaps. The same motif state gives the serving backend advisory scheduling and cache hints. Across three benchmarks, this boundary reduces LLM calls and token consumption at comparable task performance. Serving gains are complementary and depend on queueing and cache pressure. The broader lesson is to make verifiable agent structure executable, rather than repeatedly asking the model to re-construct it.

Acknowledgments

This work was supported by Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), Hubei Provincial Natural Science Foundation of China (No. 2026AFA002), and National Natural Science Foundation of China (No. 62621005). We thank our PC, shepherd, the anonymous reviewers, and the members of our research group for their valuable feedback and suggestions.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17682–17690. doi:10.1609/aaai.v38i16.29720
- [4] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 186 (2023), 24 pages. doi:10.1145/3591300
- [5] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. 2015. Pegasus, a Workflow Management System for Science Automation. *Future Generation Computer Systems* 46 (2015), 17–35. doi:10.1016/j.future.2014.10.008
- [6] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://doi.org/10.52202/079017-1882>
- [7] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034* (2024). <https://doi.org/10.48550/arXiv.2402.14034>
- [8] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338.
- [9] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 415–430. doi:10.1145/3731569.3764814
- [10] Ronald L. Graham. 1966. Bounds for Certain Multiprocessing Anomalies. *Bell System Technical Journal* 45, 9 (1966), 1563–1581.
- [11] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. 2016. GRAPHENE: Packing and Dependency-Aware Scheduling for Data-Parallel Clusters. In *Proceedings of 12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Savannah, GA, 81–97. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/grandl_graphene
- [12] Jingyi Jia and Qinbin Li. 2026. AutoTool: Efficient Tool Selection for Large Language Model Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. Association for the Advancement of Artificial Intelligence, Singapore, 31265–31273. doi:10.1609/aaai.v40i37.40389
- [13] James E. Kelley and Morgan R. Walker. 1959. Critical-Path Planning and Scheduling. In *Proceedings of the Eastern Joint Computer Conference*. 160–173. doi:10.1145/1460299.1460318
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *Proceedings of International Conference on Learning Representations*. 54928–54958. https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html
- [15] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. <https://proceedings.mlr.press/v235/kim24y.html>
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [17] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 9, 18 pages. <https://dl.acm.org/doi/10.5555/3691938.3691947>
- [18] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv preprint arXiv:2511.02230* (2025). <https://arxiv.org/abs/2511.02230v7> Version 7, revised September 8, 2026.
- [19] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116.
- [20] Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. Autoflow: Automated workflow generation for large language model agents. *arXiv preprint arXiv:2407.12821* (2024). <https://doi.org/10.48550/arXiv.2407.12821>
- [21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 929–945. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [22] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. Association for Computing Machinery, 38–56. doi:10.1145/3651890.3672274
- [23] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/871ed095b734818cfba48db6aeb25a62-Abstract-Conference.html
- [24] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2026. Agentix: An Efficient Serving Engine for LLM Agents as General Programs. In *Proceedings of 23rd*

- USENIX Symposium on Networked Systems Design and Implementation. USENIX Association, Renton, WA, 2443–2459. <https://www.usenix.org/conference/nsdi26/presentation/luo>
- [25] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Proceedings of Conference on Language Modeling*. <https://openreview.net/forum?id=L4HHkCDz2x>
- [26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [27] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.
- [28] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, Minghua Ma, Pu Zhao, Si Qin, Xiaoting Qin, Chao Du, Yong Xu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2023. TaskWeaver: A Code-First Agent Framework. *arXiv preprint arXiv:2311.17541* (2023). <https://arxiv.org/abs/2311.17541>
- [29] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2026. Mooncake: A kv-cache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* 22, 4 (2026), 1–38.
- [30] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *Proceedings of International Conference on Learning Representations*. 9695–9717.
- [31] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. Curran Associates, Inc., 68539–68551.
- [32] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/77c33e6a367922d003ff102ffb92b658-Abstract-Conference.html
- [33] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI’24). USENIX Association, USA, Article 52, 24 pages. <https://dl.acm.org/doi/10.5555/3691938.3691990>
- [34] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 31094–31116. <https://proceedings.mlr.press/v202/sheng23a.html>
- [35] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. 2025. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. In *Proceedings of International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5bc342f48de8264779952fac378f96dc-Abstract-Conference.html
- [36] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI’24). USENIX Association, USA, Article 10, 19 pages. <https://dl.acm.org/doi/10.5555/3691938.3691948>
- [37] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS ’25). Association for Computing Machinery, New York, NY, USA, 1302–1316. doi:10.1145/3676641.3716278
- [38] Xiaoyu Tan, Bin Li, Xihe Qiu, Chao Qu, Wei Chu, Yinghui Xu, and Yuan Qi. 2025. Meta-agent-workflow: Streamlining tool usage in llms through workflow construction, retrieval, and refinement. In *Companion Proceedings of the ACM on Web Conference 2025*. 458–467.
- [39] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. 2025. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229* (2025).
- [40] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274.
- [41] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 16022–16076. doi:10.18653/v1/2024.acl-long.850
- [42] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A. Kozuch, Mor Harchol-Balter, and Gregory R. Ganger. 2016. Tetrisched: global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In *Proceedings of the Eleventh European Conference on Computer Systems*. Association for Computing Machinery, 35:1–35:16. doi:10.1145/2901318.2901355
- [43] Noppanat Wadlom, Junyi Shen, and Yao Lu. 2026. Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. *Proceedings of the ACM on Management of Data* 4, 3, Article 169 (2026), 29 pages. doi:10.1145/3802046
- [44] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=ehfRiF0R3a>
- [45] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- [46] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. <https://proceedings.mlr.press/v267/wang25bx.html>
- [47] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference. In *Proceedings of 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 57–74. <https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang>

- [48] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkan Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryan W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of First Conference on Language Modeling*. <https://openreview.net/forum?id=BAakY1hNKS>
- [49] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. 2025. The Rise and Potential of Large Language Model Based Agents: A Survey. *Science China Information Sciences* 68, 2 (2025), 121101. doi:10.1007/s11432-024-4222-0
- [50] Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. 2026. GraSP: Graph-structured skill compositions for LLM agents. *arXiv preprint arXiv:2604.17870* (2026).
- [51] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 37)*. Curran Associates, Inc., 52040–52094. doi:10.52202/079017-1650
- [52] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. *arXiv preprint arXiv:2305.18323* (2023). <https://arxiv.org/abs/2305.18323>
- [53] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 37)*. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601
- [54] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery. doi:10.1145/3689031.3696098
- [55] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proceedings of International Conference on Learning Representations*. 9965–10017. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html
- [56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *Proceedings of International Conference on Learning Representations (ICLR)*.
- [57] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of 16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538.
- [58] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *Proceedings of International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html
- [59] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 38)*. Curran Associates, Inc., 114372–114398. doi:10.52202/085713-3451
- [60] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfceb2827f8-Abstract-Conference.html
- [61] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642.
- [62] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 37)*. Curran Associates, Inc., 62557–62583. doi:10.52202/079017-2000
- [63] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [64] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62138–62160. <https://proceedings.mlr.press/v235/zhou24r.html>
- [65] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of International Conference on Learning Representations*. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html